GovTech – Salesforce QA Automation Assessment

Objective

Design and implement a test automation framework and automate the test scenarios below for Salesforce CRM.

Application: Salesforce CRM (free developer signup)
https://developer.salesforce.com/signup

You should demonstrate:

- Robust UI Automation Framework using Playwright (TypeScript preferred)
- Effective use of assertions, waits, and reusable locators
- Maintainable Page Object Model (POM)
- Test stability despite dynamic elements / shadow DOM / dynamic Lightning components
- Structured reporting, logging, and config management
- Clear documentation on how to execute the tests and view the report

---

1. Navigate to the Salesforce Developer signup page.
2. Register a new Salesforce Developer account

Part A — Salesforce Login

3. Login to Salesforce after activation (manual activation can be done once; then cached storage state can be used).
4. Validate successful login by checking for the global search bar or App Launcher icon.

Part B — Test Scenario: Salesforce CRM

Lead Creation & Management

1. Go to the Sales App
2. Create a new Lead with randomly generated details:
    - First name, Last name & complete Lead information
    - Company name
    - Lead Source

3. Save the Lead.
4. Validate that:
    - The Lead has a unique Lead ID (18-digit).
    - Correct data is shown in the details view.
5. Edit the Lead → change Status (e.g. "Unqualified" → "Qualified").
6. Validate that the status updated correctly.

Lead → Opportunity Conversion (with Account & Contact Existence Check)

1. Convert the Lead — By clicking the Convert Button
2. In the conversion modal: First check
    - **If the Account already exists** — select and link to the existing Account instead of creating a new one.
    - **If the Contact already exists** — select and link to the existing Contact instead of creating a new one.
    - **If neither exists** — create both new Account and Contact, as normal.
3. Validate:
    - The Opportunity is created successfully.
    - The Account linked to the Opportunity matches the correct one — either the pre-existing record or the newly created one, depending on which branch fired.
    - The Contact linked is likewise correct for whichever branch fired.
4. Navigate to the newly created Opportunity page.
5. Validate key values (e.g. Opportunity Owner, Stage).

API / Response Validation (Bonus — shows senior-level thinking)

After conversion, capture the backend API call (XHR / network route) for the Lead → Opportunity conversion request.

- Assert at least 2 deep properties in the JSON response based on what you actually observed (e.g. an ID field exists and has the expected format; a stage/status field equals the value you saw after conversion).

Negative Scenarios

- Attempt creating a Lead with missing mandatory fields → ensure proper validation error banners are shown.
- Attempt a duplicate Lead email if the environment supports merge/duplicate detection.

Execution Design

- Use cached login (Playwright `storageState.json`).
- Parallel execution — Leads generated uniquely per test run.
- Cross-browser — Chromium + WebKit smoke run.

Part C

a. Shadow DOM / Dynamic Component Evidence

Somewhere in your framework you will hit at least one Lightning/OmniStudio element that a plain `page.locator()` call won't reach cleanly (shadow DOM piercing, dynamically generated `data-*` attributes, etc.).

For **one such element**, include in your submission:

1. A screenshot of the element's DOM structure (DevTools) showing why the naive locator didn't work.
2. The locator strategy you used instead, and a short (2–3 sentence) explanation of why it's stable.

This should come from your own org — not a generic example.

b. Handling Email Verification Code on Login

Salesforce requires a one-time email verification code on any login from an unrecognized browser/device — this includes your first Playwright-driven login, and any subsequent run where the cached session (`storageState.json`) has expired or been invalidated (e.g. password change, org security policy, cache deleted in CI).

Task:

1. Research and implement a way for your automation bypass this verification or automate the verification code retrieval and authentication using that.
2. After successful login, cache `storageState.json` so subsequent runs skip this entirely — but your solution must not assume the cache will always be valid.
3. In your README, explain your approach to solving this issue and the security/practical tradeoffs of your chosen method (e.g. using a real email account's API credentials in CI vs. a disposable inbox service, and why you picked one over the other).

Submission Requirements

1. **Git repository** with full, unsquashed commit history (not a single zip or one "initial commit"). We're interested in how the solution evolved, not just the final diff.
2. **README** with setup steps, how to run tests, and how to view the report.
3. Answers to Part C (shadow DOM evidence) and the API discovery notes from Part B, included in the README or a short `NOTES.md`.
4. A **5-minute screen-recorded walkthrough** (webcam optional) where you run the suite live and briefly narrate:
    - Where you handled a dynamic Lightning/shadow DOM component and why you chose that approach.
    - Why you chose your test data strategy (UI-based vs. API-based setup).
    - One thing that broke during development and how you debugged it.

Evaluation Focus

Area	What we're looking for
Framework design	POM structure, reusability, separation of config/test data/logic
Stability	Handling of shadow DOM, dynamic Lightning components, waits over sleeps
Reporting & logging	Structured, readable output; clear pass/fail evidence
Process evidence	Commit history, debugging narrative, README clarity